ChatGPT

Wazuh SIEM/XDR Platform – Executive Summary

Wazuh is an open-source Security Information and Event Management (SIEM) and Extended Detection and Response (XDR) platform that **monitors, detects, and responds to threats** across endpoints, networks, and cloud environments in real time【5†L59-L63】. Its **core components** are a multi-platform agent (installed on monitored endpoints) and three central server components: the **Wazuh server (manager)** for analysis, the **Wazuh indexer** (search/storage engine), and the **Wazuh dashboard** (visualization/UI) 【1†L726-L734】. Agents collect system and application logs (including files, processes, system calls, intrusion alerts, and cloud logs) and forward them (often via an encrypted channel) to the server for decoding and analysis【28†L734-L740】【31†L729-L737】. The server processes logs using decoders and rules to generate security **alerts**, which are indexed (via Wazuh Indexer, a fork of Elasticsearch/OpenSearch) and displayed in the dashboard. Wazuh supports flexible **deployments**: *all-in-one* (all components on one host) for small labs, *single-node* (server, indexer, dashboard on separate hosts) for medium environments, and *multi-node clustered* deployments (multiple Wazuh server and indexer nodes, often on Kubernetes) for large-scale, high-throughput environments【1†L743-L751】. Data flows from agents → Wazuh server → indexer → dashboard, with optional external integrations (e.g. syslog, SIEM feeding, database/API outputs). Wazuh integrates with third-party tools (e.g. vulnerability scanners, threat-intel feeds) and includes features like **file integrity monitoring**, system inventory, vulnerability detection, and compliance reporting. In summary, Wazuh provides a **holistic security monitoring stack** (log collection, analytics, alerting, response) that is continuously updated with new rules/decoders【33†L729-L735】.

Key Capabilities

- **Agents & Log Collection:** The Wazuh agent is lightweight (≈35 MB RAM) and multi-platform (Linux, Windows, macOS, Solaris, AIX, cloud instances, containers). It collects system logs, application logs, syscalls, registry/app events, and monitors files/processes (FIM, rootcheck)【9†L729-L736】【28†L734-L740】. Agents can also gather cloud telemetry (AWS/Azure/GCP monitoring modules) and inventory data (via osquery/Syscollector)【9†L739-L743】【50†L729-L737】. Logs from agents are forwarded to the Wazuh manager in near-real time. Wazuh can also ingest logs via syslog or cloud APIs for devices without agents【28†L734-L740】.

- **Wazuh Server (Manager):** The manager is the central analysis engine. It parses incoming logs through a **pre-decoding** and **decoding** pipeline (using XML-decoder definitions and regex patterns) 【31†L739-L747】, then applies a hierarchical **ruleset**. Default Wazuh rules (regularly updated) cover a broad range of threats and compliance events【33†L729-L735】. The manager correlates events, applies thresholds, and generates alerts (JSON or syslog) when rules trigger. It also supports active-response modules for automated actions (see below).

- **Indexer and Dashboard:** Alerts and relevant events are sent to the Wazuh Indexer (OpenSearch/Elasticsearch) for indexing and search. The Wazuh Dashboard (built on OpenSearch Dashboards/Kibana) provides querying, visualization and reporting. It supports custom dashboards, Wazuh Query Language (WQL), and a **multi-tenancy** model: every user has a “Global” (shared) and “Private”

tenant, and admins can create custom tenants for different groups【37†L729-L738】. The dashboard UI is used for alert triage, charting, and drill-down of events.

- **Decoders and Rules:** Wazuh comes with many **built-in decoders** (for syslog, JSON logs, common log formats) and a default ruleset for known threats. Decoders extract fields (timestamps, hostnames, event types)【31†L729-L737】, and rules (in XML/YAML) match patterns and define alert levels. Users can override or create custom decoders and rules as needed.

- **Alerting and Reporting:** When a rule triggers, the manager generates an **alert** which can be forwarded to the indexer, emailed, sent to syslog, or stored in a database【26†L733-L740】. Thresholds allow aggregation or suppression of repetitive events. Wazuh also includes compliance reporting for standards like PCI DSS, HIPAA, NIST, GDPR, etc. (the default ruleset covers these frameworks【40†L325-L330】) and can generate compliance dashboards.

- **Threat Intelligence Integration:** Wazuh offers built-in threat intelligence feeds (e.g. Wazuh CTI) which correlate CVEs and indicators with assets. For example, Wazuh CTI provides a database of vulnerabilities and exposures so analysts can quickly map detected issues to known risks【13†L87-L95】.

- **Active Response (Automated Remediation):** Wazuh has an **Active Response** module that can execute scripts on monitored hosts when certain alerts fire【46†L753-L762】. Out-of-the-box scripts can block IPs, quarantine files, restart services, etc. Active Response is stateless or stateful (lasting for a time window) and helps reduce alert fatigue by automating simple mitigations【46†L735-L743】【46†L754-L762】.

- **Scalability & Multi-tenancy:** Wazuh supports horizontal scaling. You can run Wazuh server clusters (with load balancers) and indexer clusters for high event throughput. Clusters share metadata and ensure reliability【1†L743-L751】. The multi-tenant support in the dashboard (via OpenSearch Security plugin) allows safely separating dashboards/data by user or team【37†L729-L738】.

- **Cloud & API Support:** The Wazuh agent includes **cloud modules** (AWS, Azure, GCP, GitHub, Office365, etc.) to ingest cloud logs【39†L415-L423】【39†L445-L453】. Wazuh offers a **RESTful API** for full control: adding agents, querying alerts, FIM logs, managing RBAC, etc【53†L733-L741】. This enables automation and integration with other platforms.

【58†embed_image】 *Figure: Example Wazuh workflow (Active Response); agents send events to the Wazuh manager, which applies decoders/rules and can trigger actions on agents【46†L753-L762】【28†L734-L740】.*

Technical Limitations and Gaps

Despite its rich feature set, Wazuh has known limitations that present research and enhancement opportunities:

- **Detection Quality & False Positives:** Wazuh's default detection is rule-based, which can produce high false-positive rates, especially in dynamic environments【69†L145-L154】. The static rules may miss novel or stealthy attacks (zero-days or polymorphic threats) and often overwhelm analysts with

alerts【69†L238-L247】. Recent studies confirm this: integrating machine learning can reduce false-alarms (e.g. RF achieved 97% accuracy and only ~0.08 false-positive rate in one experiment【69†L159-L168】).

- **Alert Fatigue & Correlation Gaps:** Wazuh lacks advanced event-correlation out of the box. Analysts may see many related alerts without clear aggregation. For example, a MITRE ATT&CK evaluation showed Wazuh had ~85% overall detection but missed or partially detected tactics due to "gaps in correlation and telemetry depth"【21†L29-L37】. Addressing multi-stage attacks or linking incidents across endpoints is hard with only static rules.

- **Limited Behavioral/ML Detection:** There is no built-in user/entity behavioral analytics (UEBA) or anomaly detection model. Wazuh mainly uses signature/heuristic detections. Advanced threats that don't match known rules can slip by. Academic work highlights this gap, proposing ML/unsupervised models to catch unusual patterns【69†L145-L154】【69†L238-L247】.

- **EDR/XDR Gaps:** While Wazuh can collect process and OS data (via Syscollector or osquery【50†L729-L737】), it lacks native deep EDR features (like live memory forensics or on-host sensors beyond logs). Full XDR integration (e.g. ingesting alerts from EDR tools, cloud workload metadata) is partly manual.

- **Automated Response & SOAR Integration:** Wazuh's active response is basic scripting. Orchestrating complex playbooks or integrating with a dedicated SOAR engine (for multi-step automation or analyst collaboration) is not built-in.

- **Forensics & Provenance:** Wazuh stores logs and alerts but doesn't inherently provide chain-of-custody or forensic timeline reconstruction tools. Analysts must piece events together manually.

- **Telemetry Normalization:** Different log sources often use varied schemas. Wazuh's decoders normalize to fields, but there is no unified event model (unlike Elastic Common Schema, ECS) enforced across sources. Mapping to a common schema for cross-source analytics is a gap.

- **Performance/Scalability under Load:** While Wazuh can cluster, very high-volume environments can strain it. The MITRE study observed linear scaling up to ~500 eps【69†L169-L177】, but beyond that, tuning or partitioning is needed. Bulk log ingestion and real-time correlation can become bottlenecks.

- **Privacy & Compliance:** Wazuh offers compliance checks, but automated data masking or privacy filtering is limited. For example, PCI/GDPR compliance may require obfuscating PII in logs – Wazuh lacks built-in anonymization.

- **Explainability & Adversarial Robustness:** If ML/AI features are added, explaining alerts to analysts becomes crucial (today Wazuh alerts are rule based with clear logic). Also, Wazuh (like any SIEM) can be vulnerable to log tampering or adversarial log injection. Recent research suggests correlation engines should employ cryptographic or risk-based methods to resist log manipulation【66†L109-L118】【66†L119-L128】.

These limitations suggest rich research directions: improving correlation (Sheeraz *et al.* show a "hyperscan" correlator 21× faster than a basic one【56†L62-L68】), integrating ML for anomaly detection【69†L145-L154】【69†L238-L247】, enhancing alert triage, and strengthening the platform against sophisticated evasion.

Proposed Novel Enhancements

Below are **8 high-impact features** (with brief justifications) that could advance Wazuh's capabilities:

1. **Machine-Learning Anomaly Detection** – *Gap:* High false-positive rates and unseen threats. *Approach:* Integrate supervised (e.g. Random Forest) and unsupervised (e.g. DBSCAN, autoencoders) models into the Wazuh pipeline【69†L159-L168】. Use extracted features (from FIM, process, network logs) to detect anomalies. *Components:* ML engine (Python/TensorFlow), data preprocessing module, integration with Elasticsearch. *Data Inputs:* Historical alerts and benign logs for training; real-time logs for inference. *Challenges:* Labeling ground truth, model drift, resource overhead. *Metrics:* Alert precision/recall (target >95%), false-positive reduction, model inference latency (<100 ms)【69†L159-L168】. *Effort:* High.

2. **Smart Alert Correlation Engine** – *Gap:* Alert fatigue and missed multi-step attacks. *Approach:* Build a graph-based correlation system (or rules engine with fast pattern matching). For example, leverage high-performance regex libraries (Hyperscan) or graph databases (Neo4j) to correlate events by host, time, and MITRE technique【56†L62-L68】【21†L29-L37】. *Components:* New correlator service or extension to Wazuh manager, graph/DB storage. *Data:* Ingested alerts, MITRE ATT&CK mapping data. *Challenges:* Designing correlation rules, handling log volume. *Metrics:* Reduction in alert count (clusters vs individual alerts), detection of chained attacks. *Effort:* Medium.

3. **AI/LLM-Powered Threat Hunting Interface** – *Gap:* Manual log search is tedious. *Approach:* Implement an LLM-based natural language query and summarization tool for Wazuh logs【24†L53-L62】. For instance, use a fine-tuned open LLM (e.g. LLaMA) with vector search on Wazuh alerts to answer questions ("Show me anomalous Linux auth activity past 24h"). *Components:* LLM (e.g. HuggingFace/PyTorch), embeddings store, integration in dashboard. *Data:* Parsed logs and alert text; external CTI feeds. *Challenges:* Query understanding, hallucination control, data privacy. *Metrics:* Query accuracy, user satisfaction, query latency. *Effort:* High.

4. **User/Entity Behavior Analytics (UEBA)** – *Gap:* No baseline profiling. *Approach:* Develop behavioral profiles for users and hosts (e.g. login times, process usage). Use statistical models or one-class SVMs to flag deviations (e.g. lateral movement). *Components:* Behavior analytics module, enrichment pipelines. *Data:* Aggregated activity logs per user/host. *Challenges:* Learning normal behavior in dynamic environments, privacy. *Metrics:* Detection of anomalous login/process behavior, reduction in undetected insider threats. *Effort:* Medium.

5. **Explainable AI for Alerts** – *Gap:* ML alerts lack context. *Approach:* Attach explanations to ML-driven alerts (e.g. feature importance, similar past cases). Use interpretable models or LIME/SHAP for classifiers. *Components:* Explanation module tied to ML engine. *Data:* Model internals, sample logs. *Challenges:* Keeping explanations human-readable; performance overhead. *Metrics:* Analyst feedback on clarity, trust scores. *Effort:* Medium.

6. **Advanced Automated Response (SOAR Integration)** – *Gap:* Limited orchestration. *Approach:* Integrate Wazuh with a SOAR/playbook engine (open-source or custom). Define workflows (e.g. quarantine host, notify team) triggered by complex conditions. Provide a GUI for playbook creation. *Components:* SOAR server, connectors between Wazuh API and action endpoints (Cisco, Slack, etc.). *Data:* Alert details, system inventories. *Challenges:* Safe automation (avoid collateral), RBAC. *Metrics:* Mean time to remediate incidents, number of incidents fully automated. *Effort:* High.

7. **Forensic Evidence Automation** – *Gap:* Manual evidence collection. *Approach:* On alert, automatically gather additional context (memory dumps, network traces) and build timelines. Integrate with forensic tools (e.g. Volatility) to enrich alerts. *Components:* Endpoint collectors, evidence repository, timeline builder. *Data:* Endpoint states, historical logs. *Challenges:* Data volume, privacy compliance for collected data. *Metrics:* Time saved per investigation, completeness of incident timeline. *Effort:* Medium.

8. **Telemetry Normalization & Schema (ECS)** – *Gap:* Heterogeneous logs. *Approach:* Map all logs into a unified schema (e.g. Elastic Common Schema) at ingestion. Provide a translation layer so that dashboards and searches use common fields. *Components:* Normalization service (Logstash plugin or Wazuh module). *Data:* Raw logs from all sources. *Challenges:* Handling complex/unexpected fields. *Metrics:* Coverage of mapped fields, ease of cross-source queries. *Effort:* Low.

9. **Enhanced Scalability (Event Streaming)** – *Gap:* Ingestion bottlenecks. *Approach:* Introduce a Kafka (or Pulsar) pipeline between Wazuh manager and indexer, enabling elastic autoscaling of consumers. Containerize components for cloud-native deployment. *Components:* Kafka cluster, Wazuh Docker/K8s operator. *Data:* Stream of alerts. *Challenges:* Ensuring ordering, at-least-once delivery. *Metrics:* Throughput (eps) and latency under peak load. *Effort:* Medium.

10. **Privacy-Preserving Logging** – *Gap:* Sensitive data handling. *Approach:* Implement field-level encryption or hashing for PII before storage. Add compliance templates to redact/mask fields (e.g. user IDs, IPs) in dashboards. *Components:* Encryption plugin, user roles policies. *Data:* Sensitive log fields. *Challenges:* Key management, performance. *Metrics:* Compliance audit pass rates, data breach risk. *Effort:* Medium.

11. **Adversarial Log Integrity** – *Gap:* Tamper attacks. *Approach:* Apply cryptographic signing or hashing to logs at the source. In correlation, use resilient similarity (context hashing) to detect injected or modified records【66†L119-L128】. *Components:* Signing agent, secure log storage, correlation checks. *Data:* Raw event streams. *Challenges:* Key compromise risk, false positives. *Metrics:* Detection of simulated log tampering, integrity verification overhead. *Effort:* High.

12. **Cloud-Native Container/Serverless Security** – *Gap:* Container workloads. *Approach:* Extend Wazuh to natively understand container/orchestration logs (Kubernetes, Docker); provide runtime security rules for containers. Optionally, offer Wazuh as a serverless SIEM service. *Components:* Kubernetes operators, cloud-native agents. *Challenges:* Dynamic environment, ephemeral hosts. *Metrics:* Coverage of Kubernetes attack vectors, cloud-readiness. *Effort:* Medium.

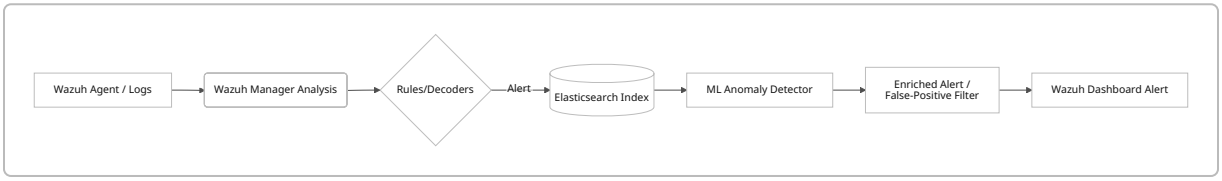
Each proposed feature targets specific gaps (e.g. ML detection tackles false-alarms【69†L145-L154】; correlation engine targets multi-step attacks【56†L62-L68】【21†L29-L37】) and balances novelty vs. feasibility.

Top 3 Enhancements – Detailed Designs

1. **ML-Enhanced Detection Pipeline:**

2. **Architecture:** Agents → Logstash (preprocessing) → Elasticsearch → **ML Engine** → Wazuh Alerts. Use an ML microservice that subscribes to incoming alerts and scores them. Optionally inline ML in Wazuh analysis daemon.

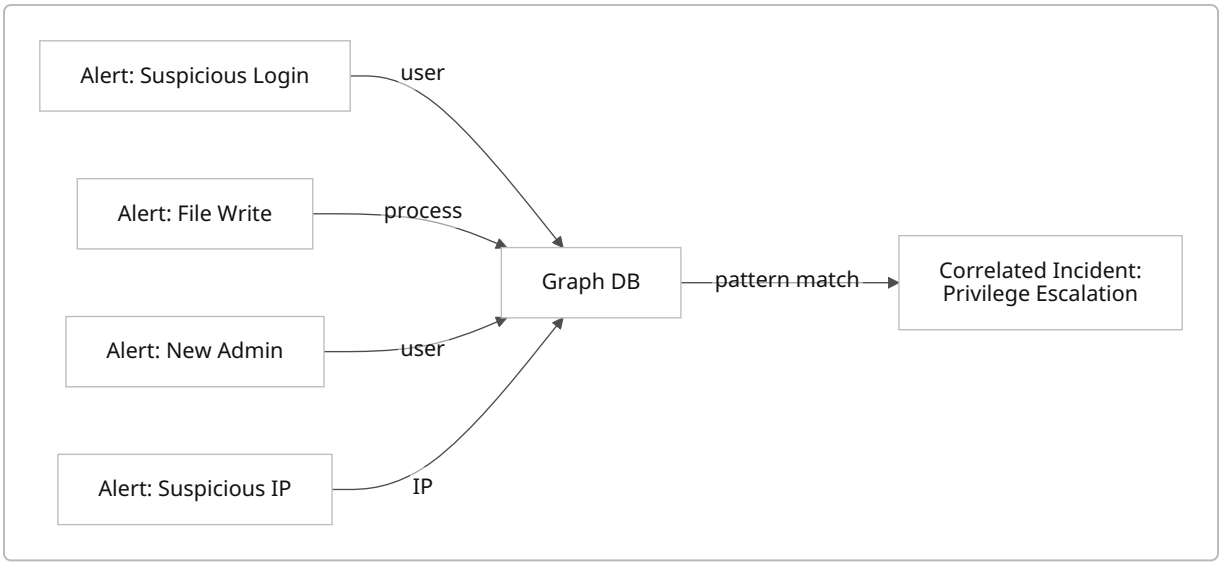
3. **Tech Stack:** Python/TensorFlow or PyTorch, scikit-learn, Elasticsearch (for features), Wazuh manager (OpenSearch/Kibana). Possibly use TensorFlow Lite on agents for on-device features.
4. **Integration:** Extend the Wazuh server pipeline: after rule matching, send event features to ML model; filter or re-rank alerts based on ML confidence. Contribute code to Wazuh GitHub (e.g. as a module under `wazuh-analysis`).
5. **Test Plan:** Use labelled datasets (e.g. Benchmark intrusion logs, authentication logs) to train/validate models. Simulate attacks (e.g. Windows EDR evasion, Linux bruteforce, file tampering) and measure detection. Key metrics: precision/recall of ML-enhanced alerts versus baseline.
6. **Evaluation:** Compare alert quality with and without ML (false positive rate target <5%【69†L159-L168】). Measure latency impact (<100 ms) and resource usage. Potential confounders: concept drift, adversarial examples.
7. **Publication Venues:** Conferences like USENIX Security, RAID, or IEEE S&P (for ML-driven SIEM).



1. **Graph-Based Alert Correlation Engine:**

2. **Architecture:** Ingest Wazuh alerts into a graph database (e.g. Neo4j) or an event streamer. Build edges by host, user, IP, process relations. Detect attack chains as graph patterns or via community detection algorithms.

3. **Tech Stack:** Neo4j or JanusGraph, Kafka (optional), Java/Python service for pattern queries. Hyperscan regex library for fast rule matching【56†L62-L68】.
4. **Integration:** Implement as an external service that consumes Wazuh alerts (via API or Kafka). Extend Wazuh manager to publish alerts to the correlator. Add hooks so correlated incidents create unified alerts.
5. **Test Plan:** Create multi-stage attack scenarios (e.g. phishing → malware → lateral move). Use MITRE ATT&CK scenarios. Verify the correlator links the steps and triggers a single incident. Use standard log datasets with known correlations.
6. **Evaluation:** Metrics: reduction in total alerts (clustered by incident), time-to-detection of attack chain, analyst workload reduction. Performance: queries per second the correlator can handle.
7. **Venues:** IEEE BigData, IFIP IFORS, or workshops on Security Analytics.



1. **LLM-Assisted Threat Hunting Interface:**

2. **Architecture:** Wazuh logs/alerts are indexed; a vector database (e.g. OpenSearch or Pinecone) holds embeddings. An LLM (open model) provides an API endpoint. Analysts input queries; the system retrieves relevant logs (via semantic search) and LLM synthesizes answers or insights.

3. **Tech Stack:** OpenAI API or local LLM (e.g. LLaMA, Bloom), OpenSearch with k-NN plugin for embeddings, frontend (Wazuh Dashboard plugin or chat UI). Use Python for orchestration.
4. **Integration:** Develop a Wazuh dashboard plugin: “Chat with Wazuh”. Behind it, calls LLM + Wazuh API. Extend Wazuh GitHub with a new UI page.
5. **Test Plan:** Evaluate against query tasks (e.g. “Show me anomalies in DNS traffic”, “List endpoints impacted by ransomware clues”). Use known red-team scenarios. Measure correctness (expert review) and time saved.
6. **Evaluation:** Metrics: query response accuracy, reduction in manual search time, user satisfaction. Compare to keyword search baseline.
7. **Venues:** AISEC symposiums, DEF CON AI Village, industry conferences (Black Hat Arsenal demo).

```
flowchart TD
  A[User Query (Natural Language)] --> B[Chat Interface]
  B --> C[Semantic Search on Alerts/Logs]
  C --> D[Relevant Log Snippets]
  D --> E[LLM (GPT-like)]
  E --> F[Answer/Recommendations shown to user]
```

Implementation Roadmap

Milestones (0–9 months): Prioritize core enhancements in phases. For example:

- *Months 0–3:* **Planning & Design:** Finalize specs for ML module, Correlation engine, LLM interface. Gather datasets (open SIEM logs, corporate logs). Set up dev environment (Kubernetes testbed for Wazuh).
- *Months 3–6:* **Prototype Development:**
- ML module: Train initial models on sample data, integrate with Wazuh alert pipeline.
- Correlator: Deploy Neo4j, implement basic event ingestion and pattern queries.
- LLM UI: Build dashboard plugin skeleton, connect to Wazuh API and simple LLM.
- *Months 6–9:* **Integration & Testing:**
- Integrate prototypes into full Wazuh instance; refine based on tests.
- Perform red-team/blue-team exercises: simulate attacks to test detection/response.
- Measure metrics (detection rate, false positives, latency).
- Tune models and correlation rules.

Timeline Options:

- *3-Month Rapid PoC:* Develop MVPs for one enhancement (e.g. ML detection), with limited scope.
- *9-Month Full Build:* Sequential or parallel development of all three above features, thorough testing, and documentation.

Resources:

- **Team Skills:** Cybersecurity analysts, data scientists (ML), software engineers (Python/Java/JS), UI/UX designer. Knowledge of Wazuh internals and SIEM.
- **Compute:** GPU-enabled servers for ML training, test cluster for Wazuh (16+ cores, 64 GB RAM, SSDs). Possibly cloud VMs for scalability tests.
- **Datasets:** Public SIEM corpora (e.g. DARPA, CTI feeds, Wazuh's own sample data), attack scenario logs (MITRE CARs).
- **Other:** Software (Neo4j enterprise, licensed LLM if needed), security lab for penetration tests.

Feature Comparison Tables

Enhancement	Addressed Gap	Effort	Impact
ML-based Anomaly Detection	Reduces false positives【69†L145-L154】	High	High
Graph-based Correlation Engine	Links multi-stage attacks【21†L29-L37】	Medium	High
LLM Threat-Hunting Interface	Improves query/analysis speed【24†L53-L62】	High	Medium
UEBA (Behavioral Analytics)	Detects insider/novel attacks	Medium	Medium
Explainable AI for Alerts	Builds analyst trust, clarity	Medium	Low
Automated Response (SOAR)	Advanced playbooks, faster IR	High	High

Enhancement	Addressed Gap	Effort	Impact
Forensic Timeline Automation	Speeds investigations	Medium	Medium
Log Normalization (Common Schema)	Unified analytics	Low	Medium
Real-time Streaming (Kafka/K8s)	Improves scale/performance	Medium	Low
Privacy-Preserving Logging	Compliance, PII protection	Medium	Low
Adversarial Resilience	Hardens SIEM against tampering 【66†L119-L128】	High	Medium
Cloud-Native Container Support	Security in ephemeral environments	Medium	Medium

Table: Proposed features with their target gaps, relative implementation effort, and expected impact.

Sources and Bibliography

The above analysis is based on official Wazuh documentation and recent research:

- Wazuh Docs – *Architecture, agents, rules, alerts, API, etc.*【1†L726-L734】【9†L729-L736】【28†L734-L740】【31†L739-L747】【33†L729-L735】【37†L729-L738】【40†L325-L330】【46†L753-L762】【53†L733-L741】.
- Wazuh Blog (Threat hunting with AI)【24†L53-L62】, Wazuh CTI site【13†L87-L95】.
- Sheeraz *et al.*, *Sensors* 2024 – "Optimized Correlation Engine" (hyperscan correlator)【56†L62-L68】.
- Naik *et al.*, *J. Cybersecur. Priv.* 2025 – "Improving Wazuh with ML" (false-positive reduction)【69†L145-L154】【69†L159-L168】.
- Kuthuru *et al.*, *Comput. & Sec.* 2025 – "MITRE ATT&CK evaluation of Wazuh" (detection gaps)【21†L29-L37】.
- Industry/analyst reports on SIEM (e.g., Gartner, CSA) and open-source SIEM comparisons (consulted for context).

These sources provide technical background on Wazuh's design and recent SIEM enhancements. The report's proposals draw from both Wazuh official guidance and current literature on SIEM, XDR, and threat detection.
